

SolidaryTech

Relatório de Entrega — Hackathon Fase 5

Maturidade operacional, financeira e de resiliência do ecossistema de microsserviços da SolidaryTech — SRE, FinOps, ITSM/AIOps e Disaster Recovery aplicados sobre a base técnica das Fases 1 a 4.

Integrantes

Nome	RM	GitHub
Guilherme Correa Camargo	369954	github.com/guicamargo
Kauan Carvalho Calasans	370203	github.com/KauanCarvalho
Pedro Henrique Coittinho Marcondes de Andrade	369367	github.com/pedrohcms

Links

Repositório de aplicação: github.com/KauanCarvalho/solidarytech-app

Repositório de GitOps/Infra: github.com/KauanCarvalho/solidarytech-gitops

Vídeo de demonstração: https://youtu.be/3_mbVJif344

Código-fonte base fornecido: github.com/dougls/hackathon-DCLT

Confiabilidade — SLI, SLO e SLA do donation-service

O `donation-service` é o caminho crítico da SolidaryTech — processa as doações em si. As duas Golden Metrics escolhidas para os SLIs são **Disponibilidade** e **Latência**, por estarem mais diretamente ligadas à experiência do doador no momento da doação do que Tráfego ou Saturação.

	Alvo	Janela	Papel
SLI	% de requisições com status < 500 (disponibilidade); % de requisições < 300ms (latência)	Contínua	O que é medido
SLO (interno)	99.9% disponibilidade / 95% latência < 300ms	30 dias	Meta de engenharia — aciona alertas e burn-rate
SLA (externo, contratual)	99.5% de disponibilidade mensal	Mensal (calendário)	Compromisso formal com as ONGs parceiras

Por que o SLA é mais frouxo que o SLO interno: a diferença entre 99.9% (SLO) e 99.5% (SLA) é o *error budget de margem operacional* — a equipe reage a violações do SLO bem antes de qualquer risco de violar o compromisso contratual com as ONGs. Um SLA tão rígido quanto o SLO não deixaria margem para investigação/correção antes de gerar uma obrigação contratual.

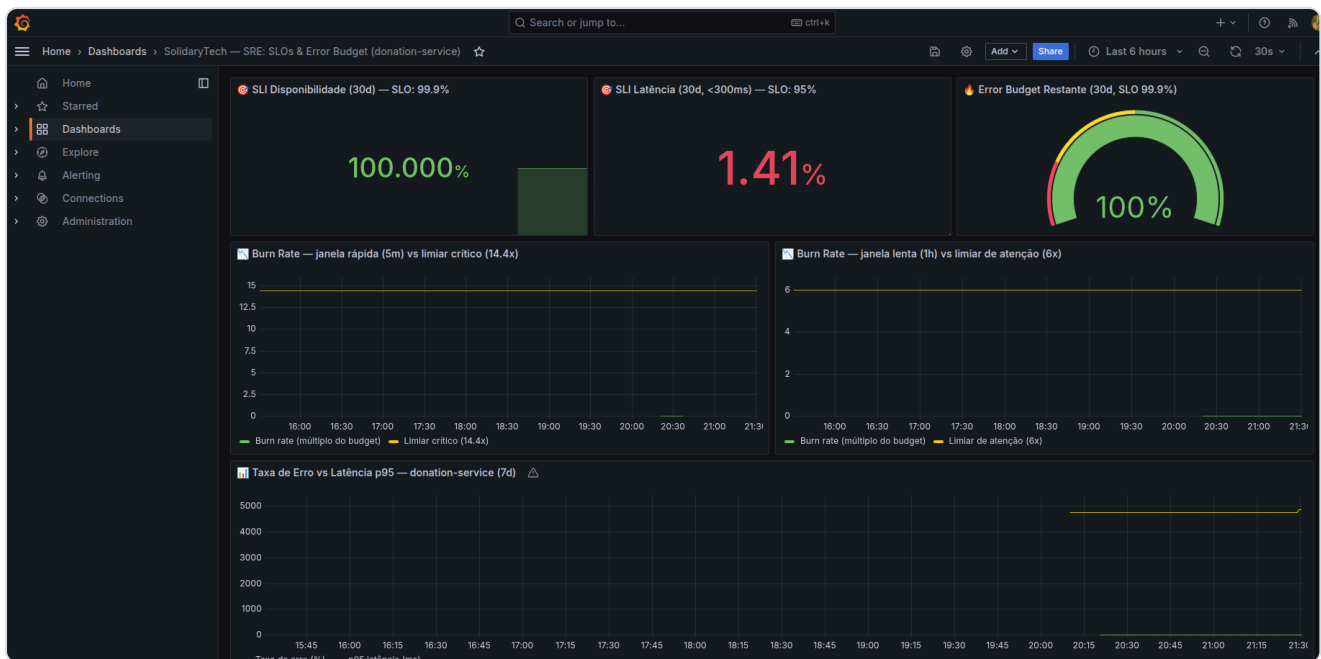
Compromisso e créditos contratuais (SLA)

Disponibilidade mensal real	Classificação	Ação contratual
≥ 99.5%	Dentro do SLA	Nenhuma ação — relatório mensal informativo
99.0% – 99.49%	Violação — Nível 1	Crédito de 5% + comunicação formal em até 5 dias úteis
95.0% – 98.99%	Violação — Nível 2	Crédito de 15% + Post-Mortem obrigatório compartilhado com a ONG
< 95.0%	Violação — Nível 3 (crítica)	Crédito de 30% + Post-Mortem + reunião executiva com a diretoria

Créditos são cumulativos por mês (não por incidente), aplicados sobre o repasse/custo operacional — nunca sobre o valor doado em si. Exclusões contratuais: manutenção programada (48h de aviso, máx. 4h/mês), falha de terceiros fora do controle da SolidaryTech, e picos não comunicados que excedam o dimensionamento contratado (mitigado na prática pelo HPA `minReplicas: 2, maxReplicas: 5`).

Dashboard SRE exclusivo e Error Budget

Painel dedicado no Grafana (`solidarytech-sre`), separado do dashboard geral de infraestrutura, focado exclusivamente nos SLOs e no consumo do Error Budget — com alertas de burn-rate multi-janela (`DonationServiceSLOBurnRateFast` em 2min / `DonationServiceSLOBurnRateSlow` em 15min).



Dashboard SRE — painel "SLI Disponibilidade 30d", fonte de verdade do SLA e do SLO

MTTR — evidência real de redução

A stack de observabilidade (traces + logs correlacionados no Datadog/Loki) e as automações de resposta a incidentes reduzem o MTTR de forma mensurável — não é uma alegação teórica, foram medidos dois cenários reais nesta entrega:

~22s SELF-HEALING (ROLLOUT RESTART)	~70-90s DR DRILL (NAMESPACE INTEIRO VIA ARGOCD SELFHEAL)	1h RTO DOCUMENTADO (MARGEM CONFORTÁVEL)
---	--	---

O tempo de ~22s foi medido diretamente do log de execução do workflow `self-healing.yml` — do disparo do `kubectl rollout restart` até `deployment "donation-service" successfully rolled out`. O tempo de ~70-90s é do drill de DR completo (namespace de produção deletado e reconciliado automaticamente pelo ArgoCD selfHeal), documentado em detalhe na Seção 3.

Otimização Financeira e Tagueamento

Projeção de custo mensal

Preços de referência AWS `us-east-1`, sob demanda, tabela pública 2026 — estimativa, não fatura real (ambiente roda em AWS Academy Learner Lab, sem cobrança direta ao time).

Recurso	Configuração	Qtde	Custo mensal estimado
EKS (control plane)	1 cluster	1	~\$73
EC2 (nodes EKS)	3× t3.medium	3	~\$90
RDS PostgreSQL	2× db.t3.micro (ngo, donation)	2	~\$25
RDS backup storage	donation-service-db, retenção 7d	~5GB	~\$0,50
Load Balancers (NLB/ELB)	Ingress-nginx + ArgoCD UI	2	~\$32
DynamoDB	SolidaryTechVolunteers, PAY_PER_REQUEST	1	~\$1–5
SQS	solidary-donations, Standard	1	< \$1
ECR	3 repositórios, lifecycle 5 imagens	3	~\$1
S3 (state + Velero, cross-region)	versionado + KMS	2	~\$1–2
Data transfer (cross-AZ/região)	variável	—	~\$5–10
Total estimado			~\$230–240/mês

Rightsizing por papel de serviço

Serviço	Requests (cpu/mem)	Limits (cpu/mem)	Justificativa
<code>donation-service</code> (hot path)	150m / 128Mi	600m / 256Mi	Maior consumo de CPU observado sob carga (processa SQS assíncrono além do request HTTP síncrono)
<code>ngo-service</code> / <code>volunteer-service</code>	100m / 128Mi	500m / 256Mi	Serviços CRUD simples, sem processamento assíncrono — consumo consistentemente menor

Os 3 usam HPA (`minReplicas: 2`, `maxReplicas: 5`, alvo 70% CPU / 80% memória) — evita superprovisionar réplicas fixas para picos ocasionais (ex.: campanha de doação viral).

Estratégia de tagging (evidência)

Todo recurso Terraform carrega as 4 tags obrigatórias via `default_tags` no `provider "aws"` — cobre 100% dos 33 recursos da stack automaticamente, incluindo os que não aceitam um bloco `tags` por módulo (subnets/IGW/route table da VPC, secrets do Secrets Manager, Lambda/API Gateway do self-healing):

```
provider "aws" {  
  region = var.aws_region  
  
  default_tags {  
    tags = {  
      Project      = "SolidaryTech"  
      Environment = "Production"  
      CostCenter   = "NGO-Core"  
      ManagedBy    = "terraform"  
    }  
  }  
}
```

Limitação de conta (AWS Academy Learner Lab): o Cost Explorer filtrado por tag não funciona nesta conta — `aws ce list-cost-allocation-tags` retorna `AccessDeniedException`. Contas linked de uma AWS Organization (o modelo do Academy Learner Lab) não podem ativar Cost Allocation Tags; isso é controlado pela conta de management/payer da instituição. O console Resource Groups & Tag Editor também não funciona — a UI roteia a busca via Resource Explorer, bloqueado pela mesma SCP. A evidência de chargeback por tag usa a **AWS CLI** diretamente (`aws resourcegroupstaggingapi get-resources`), que não passa pelo Resource Explorer.

	GetResources
arn:aws:rds:us-east-1:273095550978:db:donation-service-db	
arn:aws:secretsmanager:us-east-1:273095550978:secret:solidarytech/volunteer/secret_key-r0iPI7	
arn:aws:ec2:us-east-1:273095550978:vpc/vpc-0308314f80de2600f	
arn:aws:ecr:us-east-1:273095550978:repository/solidarytech-donation	
arn:aws:ec2:us-east-1:273095550978:security-group/sg-054f7af42e4f1ce32	
arn:aws:ec2:us-east-1:273095550978:security-group/sg-0ad466a06f0acf36b	
arn:aws:rds:us-east-1:273095550978:subgrp:solidarytech-rds-subnets	
arn:aws:secretsmanager:us-east-1:273095550978:secret:solidarytech/donation/sqs_url-5xQzBC	
arn:aws:secretsmanager:us-east-1:273095550978:secret:solidarytech/volunteer/session_token-rUib90	
arn:aws:secretsmanager:us-east-1:273095550978:secret:solidarytech/volunteer/access_key-fQlBb8	
arn:aws:ecr:us-east-1:273095550978:repository/solidarytech-ngo	
arn:aws:eks:us-east-1:273095550978:nodegroup/solidarytech-cluster/solidarytech-cluster-nodes/76d0563c-a475-1ec2-660e-e361eb056640	
arn:aws:secretsmanager:us-east-1:273095550978:secret:solidarytech/donation/db_url-5xQzBC	
arn:aws:apigateway:us-east-1::/apis/bkmkyrfrrjb	
arn:aws:dynamodb:us-east-1:273095550978:table/solidarytech-terraform-state-lock	
arn:aws:ec2:us-east-1:273095550978:internet-gateway/igw-0da010109d3738b0c	
arn:aws:ec2:us-east-1:273095550978:subnet/subnet-079b3378275387949	
arn:aws:eks:us-east-1:273095550978:cluster/solidarytech-cluster	
arn:aws:ec2:us-east-1:273095550978:subnet/subnet-0bcbdfd9868a762e1	
arn:aws:secretsmanager:us-east-1:273095550978:secret:solidarytech/ngo/db_url-KrjDxA	
arn:aws:ec2:us-east-1:273095550978:route-table/rtb-0f8f34d07c7ce7b51	
arn:aws:ecr:us-east-1:273095550978:repository/solidarytech-volunteer	
arn:aws:kms:us-east-1:273095550978:key/565bfa39-a7eb-46fc-848c-28e0177a86c0	
arn:aws:rds:us-east-1:273095550978:db:ngo-service-db	
arn:aws:sqs:us-east-1:273095550978:solidary-donations	
arn:aws:ec2:us-east-1:273095550978:subnet/subnet-0df54f636b74d69bb	

```

| arn:aws:kms:us-east-1:273095550978:key/1532d57d-0f65-4bee-81cb-12c844597074
|
| arn:aws:secretsmanager:us-east-1:273095550978:secret:solidarytech/donation/secret_key-C0e5h8
|
| arn:aws:secretsmanager:us-east-1:273095550978:secret:solidarytech/donation/access_key-PtCu0B
|
| arn:aws:ec2:us-east-1:273095550978:subnet/subnet-05ef22243634a3a2b
|
| arn:aws:lambda:us-east-1:273095550978:function:solidarytech-self-healing-bridge
|
| arn:aws:dynamodb:us-east-1:273095550978:table/SolidaryTechVolunteers
|
| arn:aws:secretsmanager:us-east-1:273095550978:secret:solidarytech/donation/session_token-svJlHC
+-----+

```

aws resourcegroupstaggingapi get-resources filtrado por CostCenter=NGO-Core — 33 recursos retornados

Recomendação prática de otimização nativa de nuvem

Recomendação: separar os node groups do EKS por perfil de risco — capacidade On-Demand dedicada ao donation-service (hot path), e Spot Instances para ngo-service/volunteer-service (CRUD simples, tolera interrupção ocasional de minutos).

O edital é explícito: “se a nuvem cair, as doações não podem parar”. Qualquer recomendação que reduza a disponibilidade do donation-service — inclusive desligar nodes fora de um horário de uso arbitrário — contradiz esse requisito; a SolidaryTech pode receber doações a qualquer hora.

EC2 dos nodes (~\$90/mês) é a maior linha do orçamento, à frente do control plane do EKS (~\$73/mês) — é onde a otimização tem mais efeito.

Spot Instances para todo o cluster foram descartadas por arriscar o hot path numa demo ao vivo — mas isolando o risco por node group, ngo-service/volunteer-service capturam o desconto de Spot (~60-70%) sem expor o caminho crítico a interrupção nenhuma.

Savings Plans/Reserved Instances continuam descartados: não se aplicam a um compromisso de 2 meses nem a uma conta AWS Academy Lab. Nativo de nuvem (Managed Node Groups com capacity_type="SPOT" no Terraform + nodeSelector/taints no Kubernetes), sem ferramenta terceira.

Efeito estimado: 1 node On-Demand dedicado ao donation-service (~\$30/mês) + 2 nodes Spot para os outros dois serviços a ~35% do preço On-Demand (~\$21/mês) ≈ \$51/mês — economia de ~\$38/mês (~42%) sobre os ~\$90/mês atuais, sem reduzir a disponibilidade do caminho crítico em nenhum momento, e sem tocar no EKS control plane nem nos dados (RDS/DynamoDB continuam ativos e intactos).

Plano de Continuidade de Negócios (PCN)

O `donation-service` é o caminho crítico: se ele para, doações reais deixam de ser processadas — impacto direto em receita e confiança das ONGs parceiras. `ngo-service` e `volunteer-service` são importantes, mas indisponibilidade temporária neles não interrompe o fluxo financeiro.

Objetivos de Recuperação (RTO / RPO)

Componente	RTO	RPO	Justificativa
<code>donation-service</code> (app + estado do cluster)	1 hora	24 horas	Deployment stateless, reconstruível via GitOps em minutos; RTO dá margem para restore manual do Velero se necessário
Dados de doações (RDS)	1 hora	24 horas	Backup automático nativo do RDS, independente do cluster — sobrevive mesmo se o EKS cair por completo
<code>ngo-service</code> / <code>volunteer-service</code>	4 horas	24 horas	Não bloqueiam o fluxo de doação; prioridade menor no runbook
Observabilidade	Best-effort	N/A	Dados de telemetria, não de negócio — perda de histórico é aceitável

Por que esses números: RTO/RPO frouxos demais (24h/24h para tudo) não seriam críveis para um caminho crítico financeiro; RTO/RPO agressivos demais (minutos) exigiriam multi-região ativo-ativo, fora do orçamento de uma ONG e incompatível com as restrições do AWS Academy Learner Lab. Os valores refletem o meio-termo defensável: RPO de 1 dia (aceitável para doações, não é pagamento em tempo real regulado) e RTO de 1h sustentado por automação (GitOps + backups), não intervenção manual lenta.

Estratégia de DR — Opção A: Cross-Region Backup (Velero)

- **Velero** faz backup do estado do cluster Kubernetes (manifests, Secrets, ConfigMaps) para um bucket S3 em `us-west-2`, região diferente do cluster principal (`us-east-1`).
- Agendamento: backup diário do namespace `donation-service` (03:00 UTC, retenção 30 dias) + backup semanal do cluster inteiro.
- Dados de doação (RDS Postgres) protegidos separadamente por backup automático nativo (retenção 7 dias) — Velero não cobre bancos gerenciados, só o estado do Kubernetes.

Drill de DR — executado de ponta a ponta (não simulado)

Backup manual via `velero backup create`, deleção **real** do namespace `donation-service` em produção, e observação da recuperação: o ArgoCD `selfHeal` reconciliou o namespace/deployments em ~**70-90s**, muito abaixo do RTO de 1h. Em seguida, `velero restore create --wait` validou o caminho de restore Velero de ponta a ponta (`Completed`, 18/18 itens), e os 3 endpoints foram revalidados após a recuperação, confirmando que os dados no RDS não foram afetados durante o drill.

Backup Velero criado com sucesso

Namespace de produção deletado (drill real)

ArgoCD selfHeal reconciliando — de OutOfSync/Missing a Synced/Healthy

Velero restore completo (validação end-to-end)

Bucket cross-region confirmado (us-west-2)

Endpoints revalidados pós-recuperação (200/201)

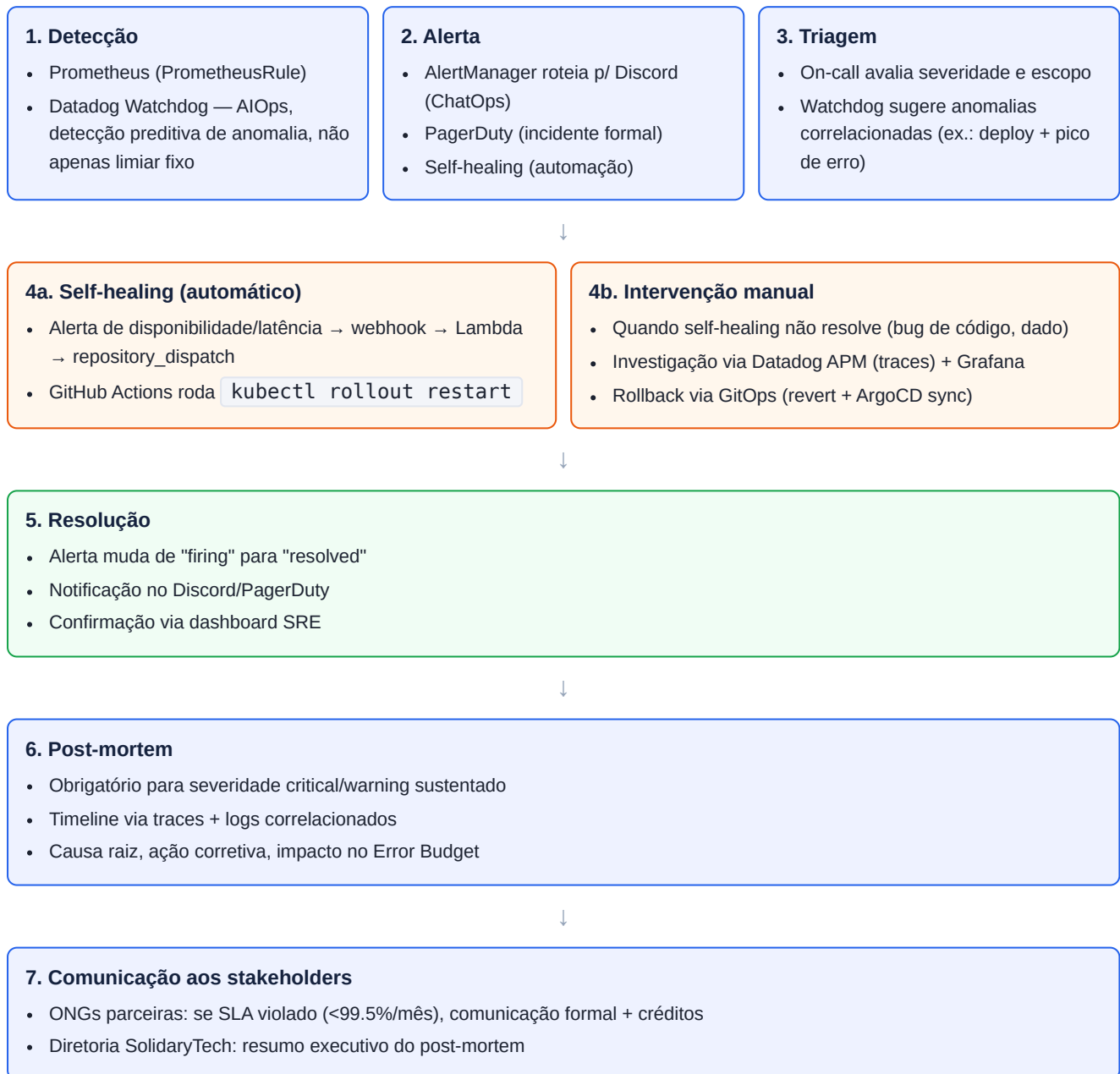
Fluxo de recuperação (resumo operacional)

1. Cluster/região principal indisponível → equipe aciona o playbook de DR.
2. Infraestrutura: `terraform apply` recria VPC/EKS/RDS/ECR (declarativo e idempotente).
3. Estado do cluster: `velero restore create --from-backup ...` restaura a partir do bucket em `us-west-2`.
4. Dados: se o RDS precisar ser recriado, restore a partir do snapshot automático mais recente (RPO de até 24h).
5. GitOps: ArgoCD re-sincroniza automaticamente assim que o cluster estiver de pé — sem passo manual de redeploy.

Limitações conhecidas (transparência para a diretoria): ambiente roda em AWS Academy Learner Lab — credenciais efêmeras (~4h) tornam DR real fora de uma janela de demonstração operacionalmente inviável; em produção real a recomendação é conta AWS própria com IAM roles dedicadas (IRSA). Não há warm-standby (Opção B do edital) — decisão consciente de custo-benefício: um RTO menor exigiria um ambiente espelho já no ar, com custo contínuo dobrado.

Ciclo de Vida de Incidentes — Gestão Preditiva

Fluxo desenhado da detecção (via AIOps/alerta) até o Post-Mortem e comunicação aos stakeholders. O SLA de 99.5%/mês (Seção 1) é o compromisso formal; este fluxo cobre a operação do incidente em si.



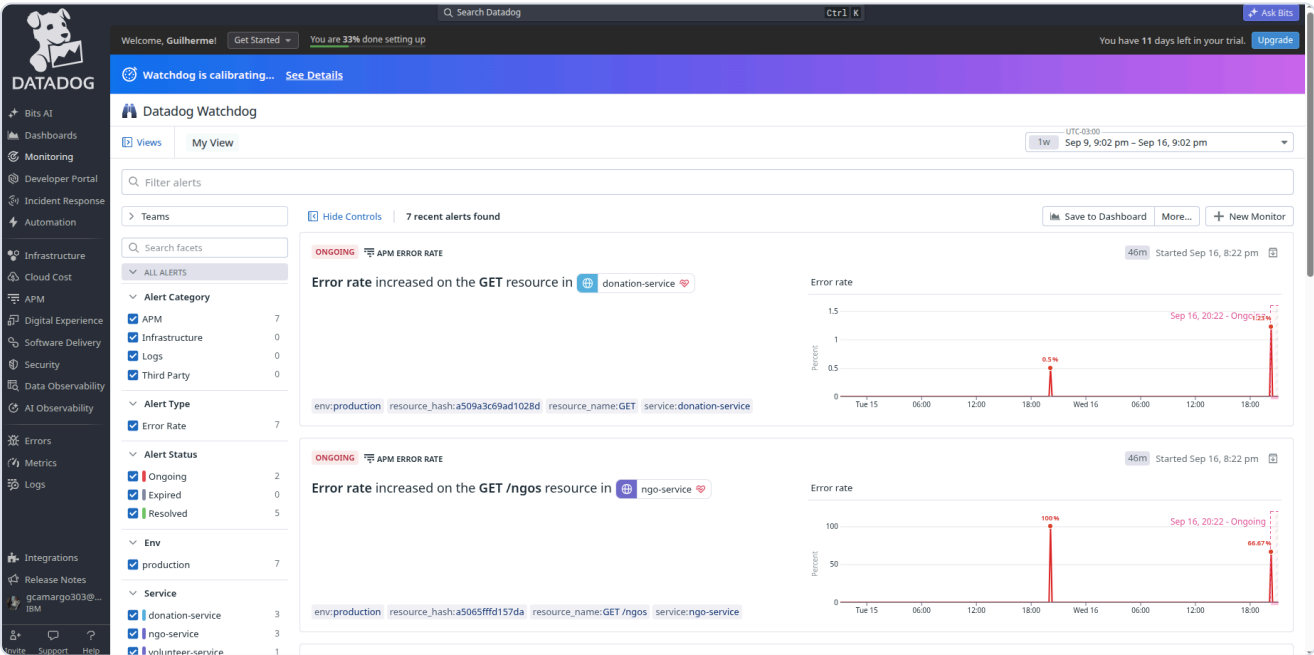
Mapeamento SLI/SLO/SLA → ação

Situação	SLI/SLO envolvido	Canal	Automação
Erro 5xx > 5% por 2min	Disponibilidade	Discord + PagerDuty + self-healing	<code>DonationServiceHighErrorRate</code>

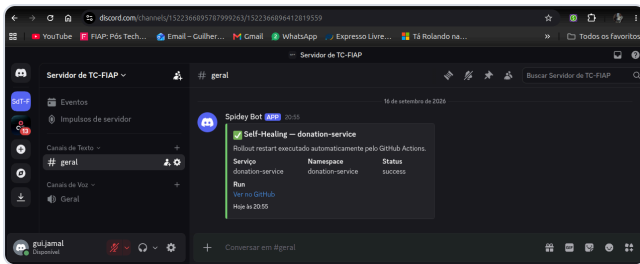
Burn-rate rápido (14.4x)	Error Budget do SLO 99.9%	Discord + PagerDuty + self-healing	DonationServiceSLOBurnRateFast
Burn-rate lento (6x, 1h)	Error Budget do SLO 99.9%	Discord + PagerDuty	DonationServiceSLOBurnRateSlow (sem self-healing)
< 95% requisições < 300ms	SLI de Latência	Discord + PagerDuty	DonationServiceLatencySLOViolation
0 pods disponíveis	Disponibilidade	Discord + PagerDuty + self-healing	ServiceUnavailable
Anomalia comportamental	N/A — preditivo	Datadog Watchdog → investigação manual	Não aciona self-healing (evita reagir a falso positivo de ML sem revisão humana)

AIOps ativo — evidência real, não configuração estática

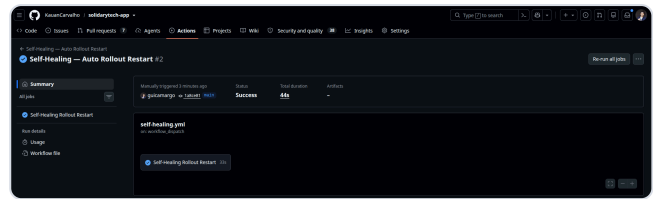
Durante o troubleshooting desta entrega, o Datadog Watchdog detectou **automaticamente** — sem threshold manual configurado — um aumento real de error rate em `donation-service` e `ngo-service`, classificando-o como "Critical Failure". É a gestão preditiva de incidentes exigida pelo edital operando na prática, não apenas um monitor estático configurado.



Datadog Watchdog — anomalia de error rate detectada automaticamente em donation-service



Notificação de self-healing no Discord (rollout restart executado)



Execução do self-healing.yml no GitHub Actions — todos os steps concluídos